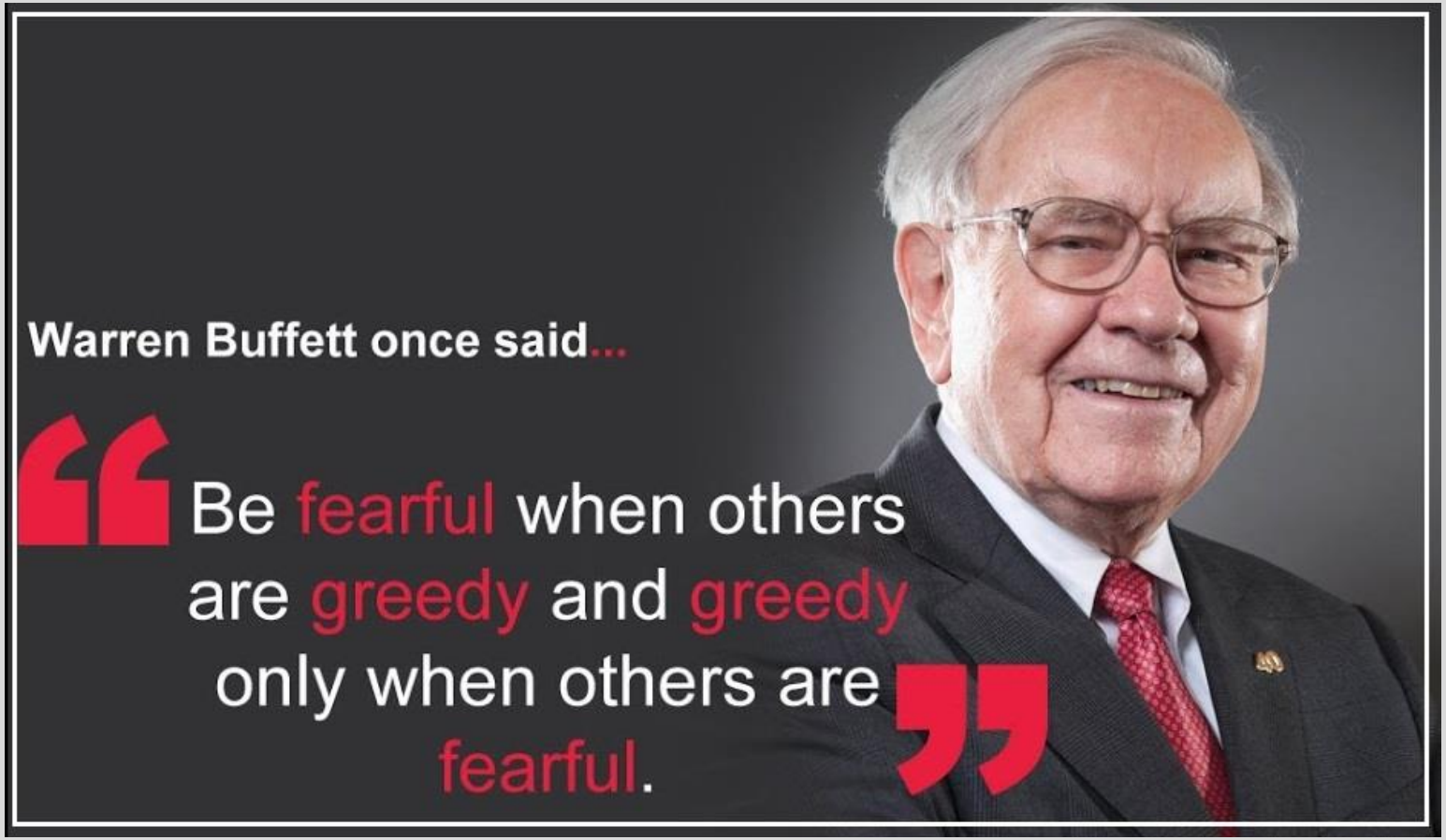
Taller de Programación Competitiva UMSS 2024

Greedy

Mateo Valdivia

cf : el_programador


Warren Buffett once said...
“Be fearful when others are greedy and greedy only when others are fearful.”

Qué es un algoritmo Greedy?

- No es un algoritmo específico.
- Se usan heurísticas para elegir la opción en cada paso local.
- Nunca doy pasos atrás en decisiones que ya tomé, por lo tanto suelen ser eficientes en Complejidad Temporal.
- A veces es difícil demostrar si realmente funciona (Probar por AC?).

KRUSKAL es un algoritmo Greedy!!!

Motivación

Dado un conjunto de cortes de billetes, encontrar la menor cantidad de billetes necesarios para formar una suma S.

Billetes = {1,2,5,10,20,50,100,200}

Para **suma = 520** la solución óptima es **{ 200, 200, 100, 20 }**

Problema 0

N niños que quieren subir a góndolas, y tu tarea es asignar una góndola para cada niño. Cada góndola puede llevar uno o dos niños, y además, **el peso total en una góndola no puede exceder x**. Conoces el peso de cada niño. ¿Cuál es el número mínimo de góndolas necesarias para los niños?

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x \leq 10^9$
- $1 \leq p_i \leq x$

Example

Input:

```
4 10
7 2 3 9
```

Output:

```
3
```

https://cses.fi/problemset/task/1090

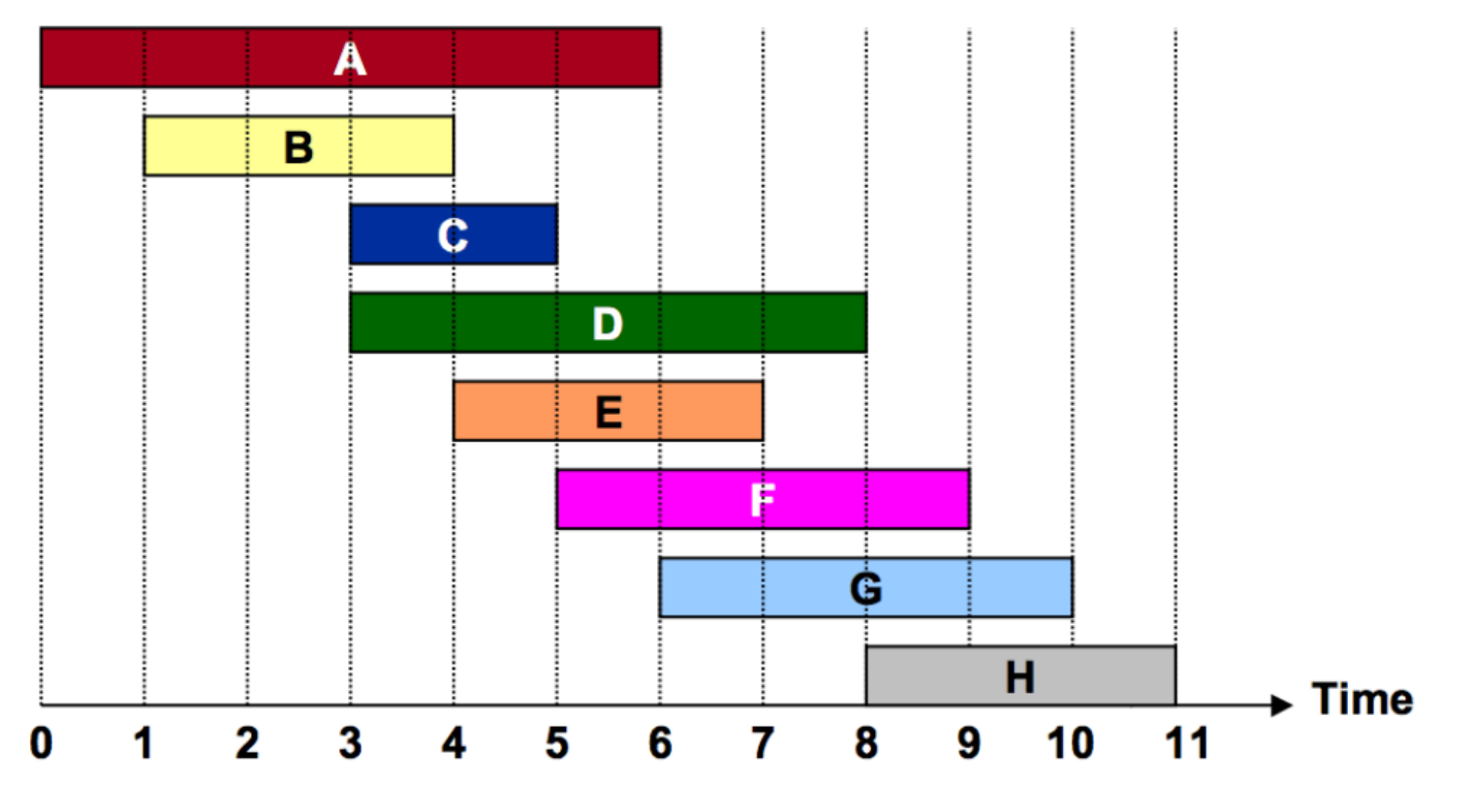
Problema 0

```
int n,x,p;
cin>>n>>x;
vi v;
for(int i=0;i<n;i++){
    cin>>p;
    v.pb(p);
}
sort(v.begin(),v.end());
int b=0,e=n-1;
int res=0;
while(b<=e){
    res++;
    if(v[b]+v[e]<=x){
        b++;e--;
    }else{
        e--;
    }
}
cout<<res<<endl;
```

https://cses.fi/problemset/task/1090

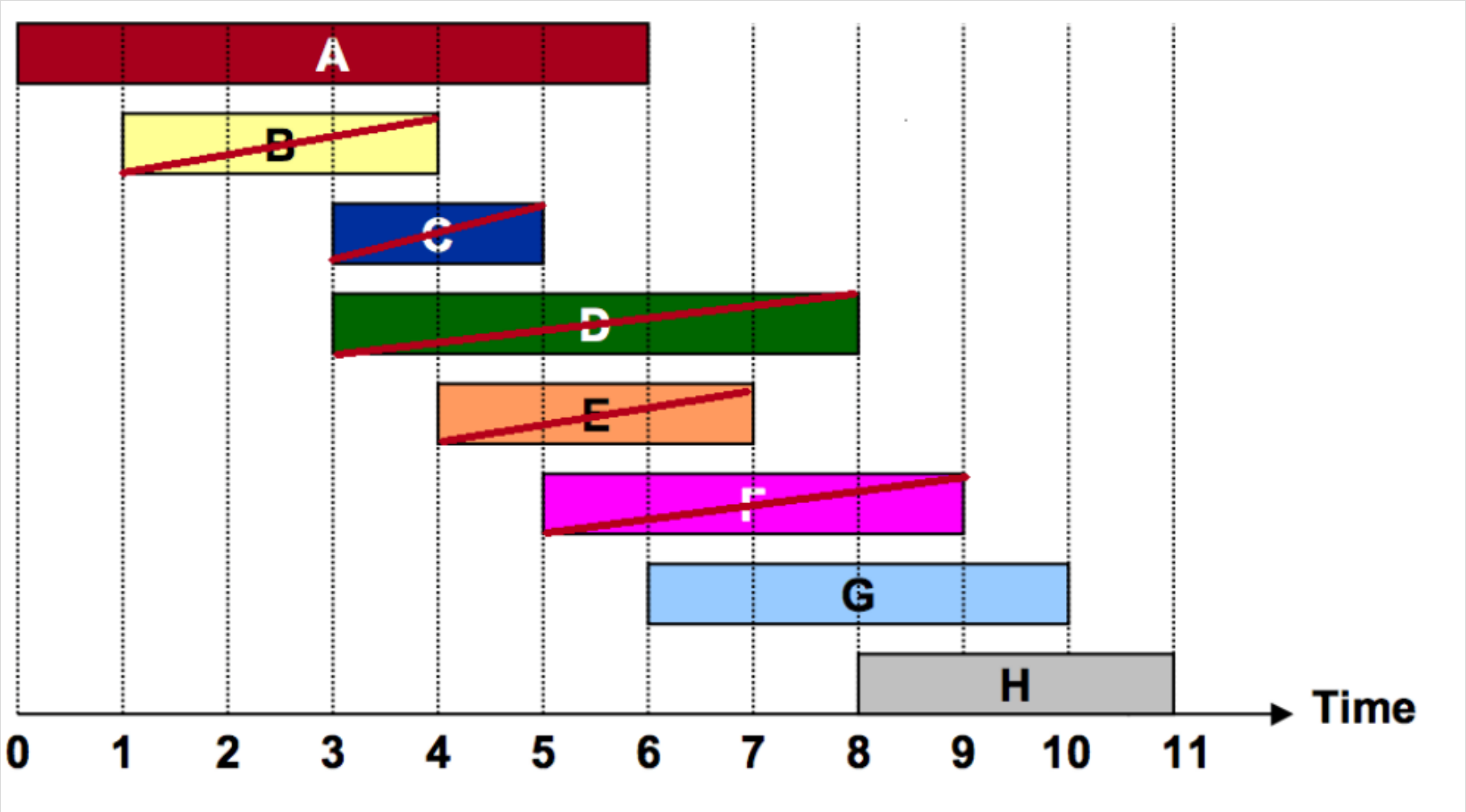
Problema de la Selección de Tareas (Problema Clásico)

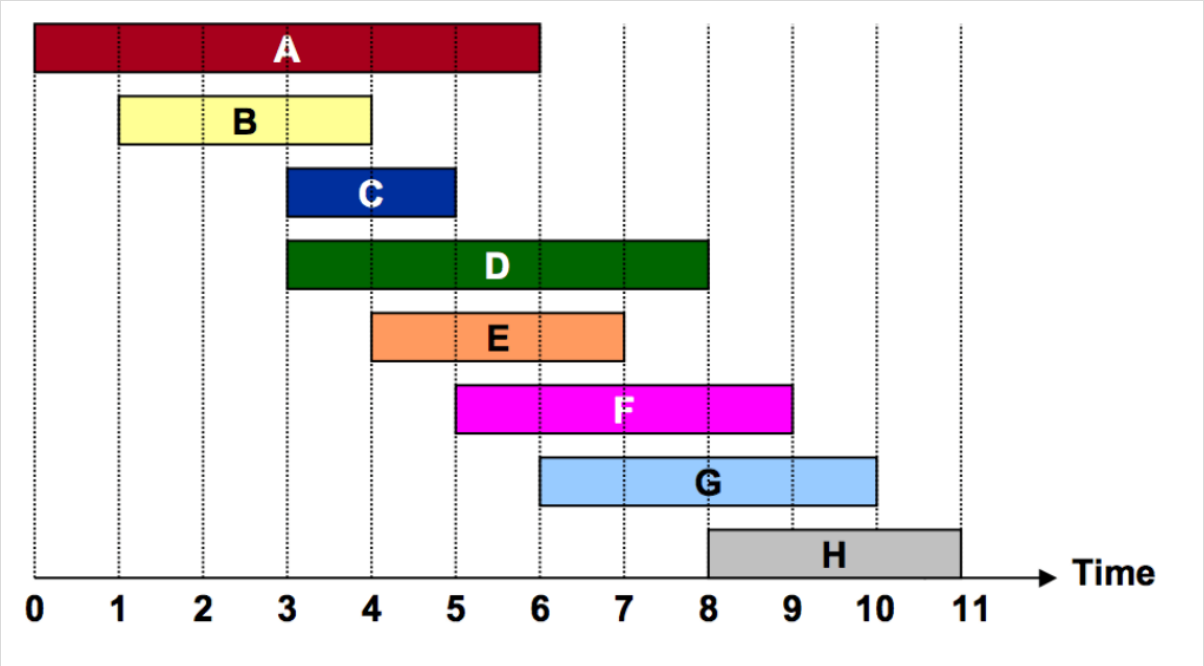
Tenemos actividades a realizar, dichas actividades tienen un principio y fin conocido. Las actividades se pueden superponer. El problema pide la máxima cantidad de actividades que podemos elegir sin que se superpongan.



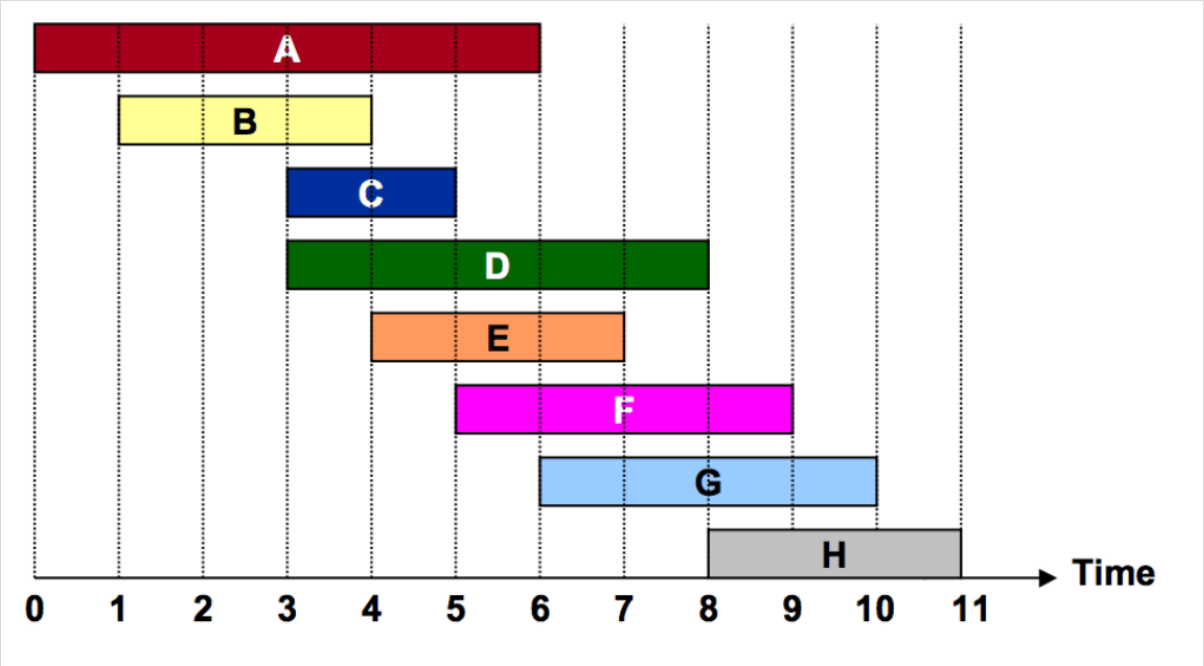
https://leetcode.com/problems/non-overlapping-intervals/description/

Problema de la Selección de Tareas

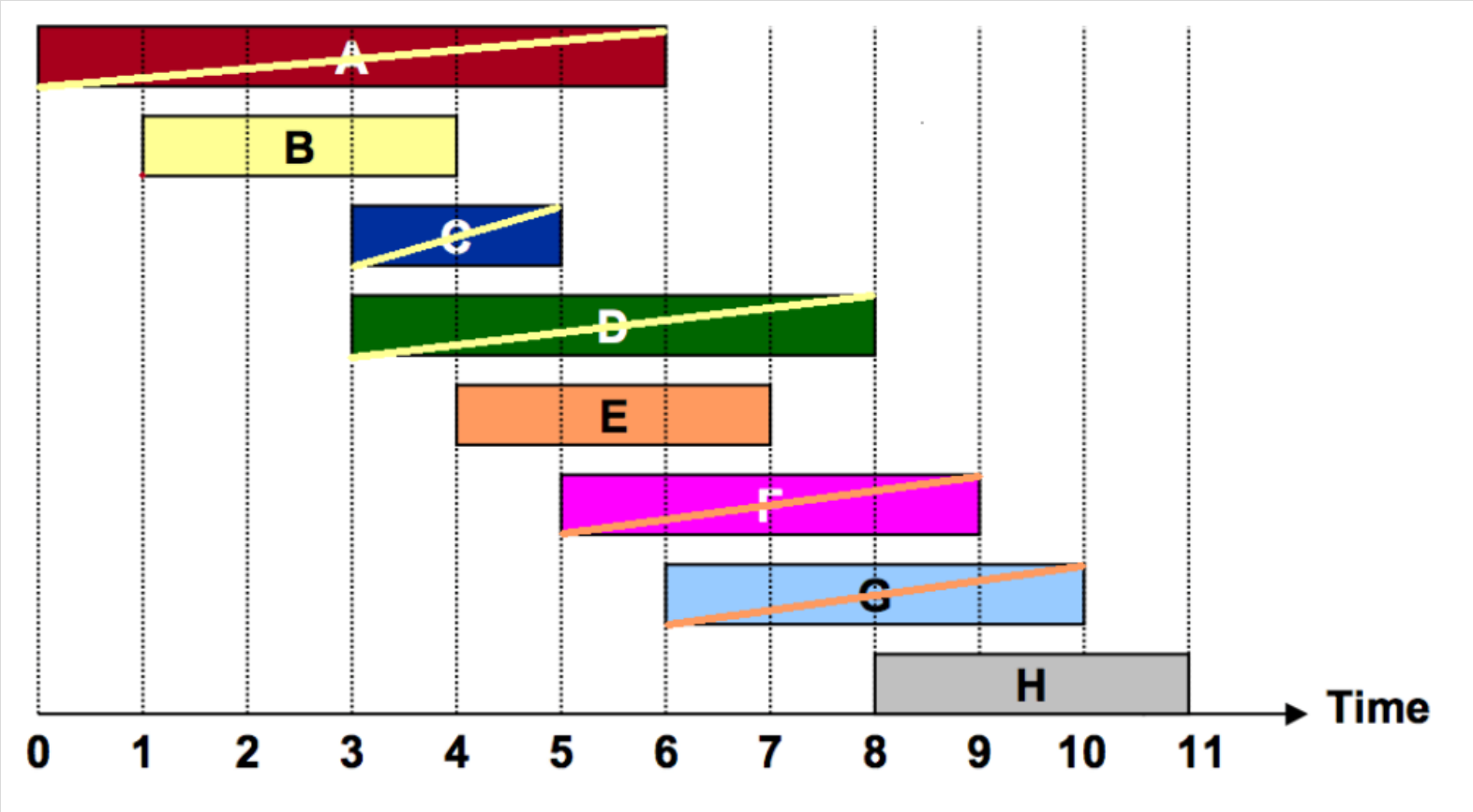




- Qué estrategia podemos usar?
- Que heurística consistente usamos?
- Cuál es la opción óptima en cada paso local?



- Impulsivo. Tomar las tareas que inician antes.
- Evitar Colisiones. Intentar tomar las tareas más cortas.
- Terminar rápido. Tomar la tarea que termina antes.



- Terminar rápido. Tomar la tarea que termina antes.

Problemas Similares – orden sugerido

https://leetcode.com/problems/non-overlapping-intervals/description/

https://cses.fi/problemset/task/1630

https://cses.fi/problemset/task/1632

https://cses.fi/problemset/task/1140

https://youkn0wwho.academy/topic-list/activity_selection_problem (más problemas)

https://codeforces.com/gym/103640/problem/M

Pregunta para la Clase

Hallar dos elementos a, b.

- a+b=n
- Maximizar a*b

Greedy Sorting (Problema Motivación)

You're given a list of n strings $a_1, a_2, \ldots, a_n$. You'd like to concatenate them together in some order such that the resulting string would be lexicographically smallest.

Given the list of strings, output the lexicographically smallest concatenation.

Input

The first line contains integer n — the number of strings ($1 \le n \le 5 \cdot 10^4$).

Each of the next n lines contains one string a_i ($1 \le |a_i| \le 50$) consisting of only lowercase English letters. The sum of string lengths will not exceed $5 \cdot 10^4$.

Output

Print the only string a — the lexicographically smallest string concatenation.

input Copy

```
4
abba
abacaba
bcd
er
```

output Copy

```
abacabaabbabcder
```

https://codeforces.com/contest/632/problem/C

Cómo hacer mi propio comparador en c++?

Ejemplo: Ordenar de mayor a menor

```
// si cmp es verdadero, entonces a va antes que b
bool cmp(int a, int b){
    if(a>b)return true;
    return false;
}
```

```
// si cmp es verdadero, entonces a va antes que b
bool cmp(int a, int b){
    return a > b;
}
```

```
vector<int> v = {3,2,4,1,5,2,3};
sort(v.begin(), v.end(), cmp);
```

Expression	Return type	Requirements
`comp(a, b)`	implicitly convertible to `bool`	Establishes strict weak ordering relation with the following properties: • For all a, `comp(a, a) == false`. • If `comp(a, b) == true` then `comp(b, a) == false`. • if `comp(a, b) == true` and `comp(b, c) == true` then `comp(a, c) == true`.

CUIDADO!!!

Ejemplo: Ordenar de mayor a menor un arreglo

```
bool cmp(int a, int b){
    return a >= b;
}
```

Expression	Return type	Requirements
`comp(a, b)`	implicitly convertible to `bool`	Establishes strict weak ordering relation with the following properties: • For all a, `comp(a, a) == false`. • If `comp(a, b) == true` then `comp(b, a) == false`. • if `comp(a, b) == true` and `comp(b, c) == true` then `comp(a, c) == true`.

CUIDADO!!!

Ejemplo: Ordenar de mayor a menor un arreglo

```
bool cmp(int a, int b){
    return a >= b;
}
```

- For all a, `comp(a, a) == false`.

Greedy Sorting (Problema Motivación)

You're given a list of n strings $a_1, a_2, \ldots, a_n$. You'd like to concatenate them together in some order such that the resulting string would be lexicographically smallest.

Given the list of strings, output the lexicographically smallest concatenation.

Input

The first line contains integer n — the number of strings ($1 \le n \le 5 \cdot 10^4$).

Each of the next n lines contains one string a_i ($1 \le |a_i| \le 50$) consisting of only lowercase English letters. The sum of string lengths will not exceed $5 \cdot 10^4$.

Output

Print the only string a — the lexicographically smallest string concatenation.

input Copy

```
4
abba
abacaba
bcd
er
```

output Copy

```
abacabaabbabcder
```

https://codeforces.com/contest/632/problem/C

Greedy Sorting (Problema Motivación)

input Copy

```
4
abba
abacaba
bcd
er
```

output Copy

```
abacabaabbabcder
```

input Copy

```
5
x
xx
xxa
xxaa
xxaaa
```

output Copy

```
xxaaaxxaaxxaxxx
```

input Copy

```
3
c
cb
cba
```

output Copy

```
cbacbc
```

SOLUCIÓN

```
bool cmp(string a, string b){
    return (a+b)<(b+a);
}
```

POR QUÉ?

SOLUCIÓN

```
bool cmp(string a, string b){
    return (a+b)<(b+a);
}
```

POR QUÉ?

EJEMPLO 2

You administer a large cluster of computers with hard drives that use various file system types to store data. You recently decided to unify the file systems to the same type. That is quite a challenge since all the drives are currently in use, all of them are filled with important data to the limits of their capacities, and you cannot afford to lose any of the data. Moreover, reformatting a drive to use a new file system may significantly change the drive's capacity. To make the reformat possible, you will have to buy an extra hard drive. Obviously, you want to save money by minimizing the size of such extra storage.

You can reformat the drives in any order. Prior to reformatting a drive, you must move all data from that drive to one or more other drives, splitting the data if necessary. After a drive is reformatted, you can immediately start using it to store data from other drives. It is not necessary to put all the data on the same drives they originally started on – in fact, this might even be impossible if some of the drives have smaller capacity with the new file system. It is also allowed for some data to end up on the extra drive.

As an example, suppose you have four drives A, B, C, and D with drive capacities 6, 1, 3, and 3 GB. Under the new file system, the capacities become 6, 7, 5, and 5 GB, respectively. If you buy only 1 GB of extra space, you can move the data from drive B there and then reformat drive B. Now you have 7 GB free on drive B, so you can move the 6 GB from drive A there and reformat drive A. Finally, you move the six total gigabytes from drives C and D to drive A, and reformat C and D.

The input begins with a line containing one integer n ($1 \leq n \leq 10^6$), which is the number of drives in your cluster. Following this are n lines, each describing a drive as two integers a and b, where a is the capacity with the old file system and b is the capacity with the new file system.

All capacities are given in gigabytes and satisfy $1 \leq a, b \leq 10^9$. (One thousand petabytes should be enough for everyone, right?)

SOLUCIÓN

```
#define ii pair<int,int>
#define F first
#define S second

bool cmp(ii A, ii B){
    if(A.F<=A.S && B.F<=B.S)return A.F<B.F;
    if(A.F<=A.S)return true;
    if(B.F<=B.S)return false;
    if(A.S>B.S)return true;
    return false;
}
```

SOLUCIÓN

```
#define ii pair<int,int>
#define F first
#define S second

bool cmp(ii A, ii B){
    if(A.F<=A.S && B.F<=B.S)return A.F<B.F;
    if(A.F<=A.S)return true;
    if(B.F<=B.S)return false;
    if(A.S>B.S)return true;
    return false;
}
```

https://codeforces.com/gym/101242 (Problema L)

TIP del pichón

Cuando tu greedy sort se puede dividir en dos grupos grandes y hallas un ordenamiento para un grupo.

OJO! puede que el problema sea simétrico y el ordenamiento sea lo mismo para el otro grupo, solo que al revés aunque no parezca.
Inténtalo te cuesta por lo mucho un WA

https://atcoder.jp/contests/abc167/tasks/abc167_f?lang=en

(Problema H ICPC World Finals 2022 es el mismo)

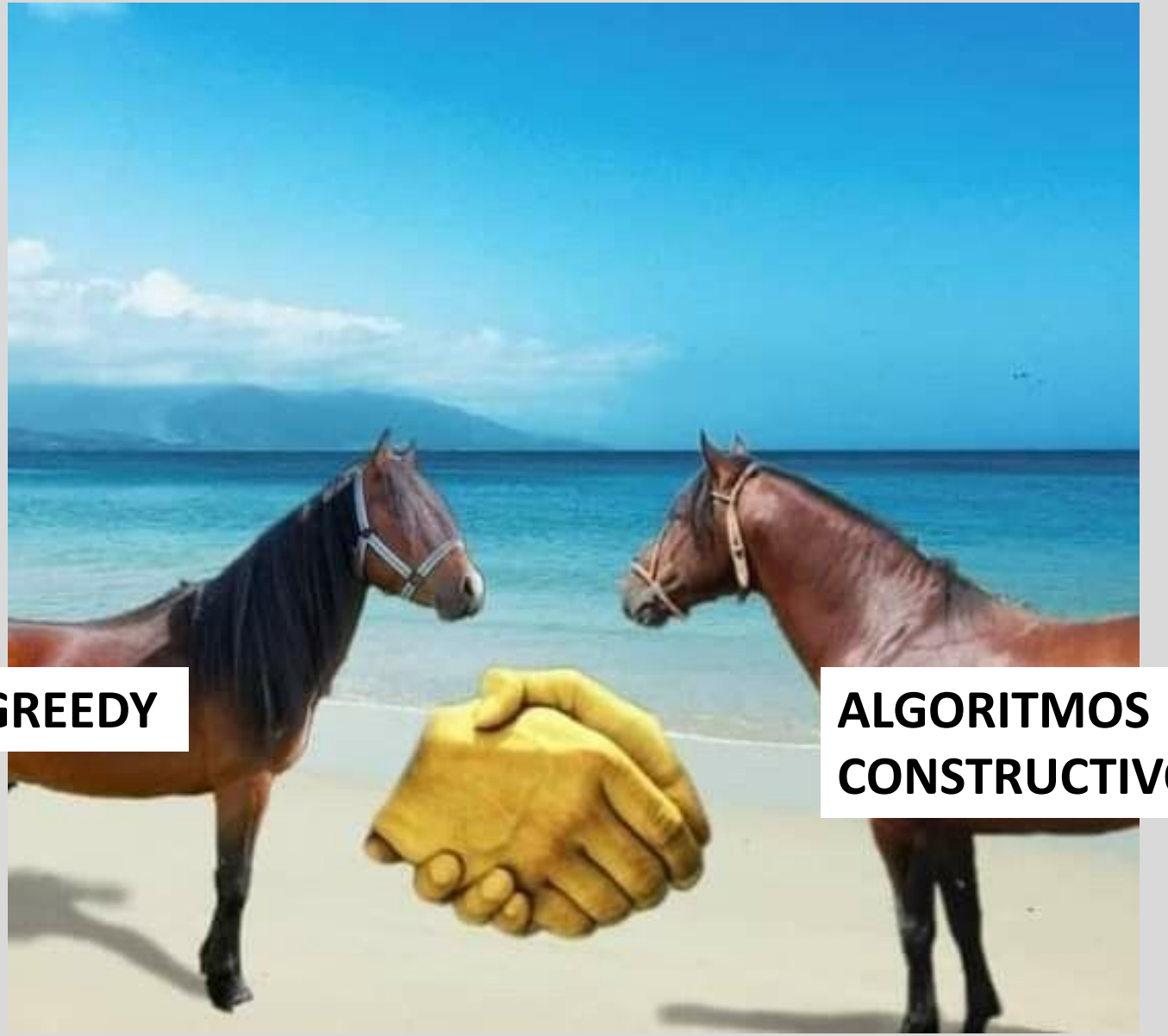
Problemas Similares – greedy sort

https://codeforces.com/gym/101341/problem/A

https://codeforces.com/gym/102916/problem/K

https://acm.timus.ru/problem.aspx?space=1&num=1745

https://atcoder.jp/contests/abc167/tasks/abc167_f?lang=en


GREEDY
ALGORITMOS CONSTRUCTIVOS

Tienes una permutación de **n** números **[1,2,3,…,n].** Hallar un **subset de k números** que sumen **S.**

n<=100000, k<=n $s\ (1 \leq s \leq \frac{n(n+1)}{2})$.

Ejemplo:

- **N = 5 [1,2,3,4,5]**
- **S = 8**

- **Respuesta = [1,2,5]**

https://codeforces.com/contest/1512/problem/E

Para una permutación con N números. [1,2,3…,N-2,N-1,N]

- Suma más bajo que puedo formar con k números es $\frac{k*(k+1)}{2}$
- Suma más alta que puedo formar con k números es $\frac{k*(2n-k+1)}{2}$

https://codeforces.com/contest/1512/problem/E

Para una permutación con N números. [1,2,3…,N-2,N-1,N]

- Suma más bajo que puedo formar con k números es $\frac{k*(k+1)}{2}$
- Suma más alta que puedo formar con k números es $\frac{k*(2n-k+1)}{2}$
- También puedo formar cualquier suma entre $[\frac{k*(k+1)}{2}, \frac{k*(2n-k+1)}{2}]$

https://codeforces.com/contest/1512/problem/E

- Puedo formar cualquier suma entre $[\frac{k*(k+1)}{2}, \frac{k*(2n-k+1)}{2}]$

Truco del pasito seguro: intento tomar el más grande (n), entonces **k:k-1, s:s-n y n:n-1**. Solo puedo tomar el más grande (n) si con mis nuevos valores de **k, s y n** sigo satisfaciendo lo que dijimos al inicio.

Si satisface entonces lo tomó y continuo buscando recursivamente, si no satisface entonces estoy seguro que este n no es parte de mi respuesta y continuo buscando de forma recursiva.

https://codeforces.com/contest/1512/problem/E

Taller de Programación Competitiva UMSS 2024

Gracias

Mateo Valdivia

cf : el_programador